

# **BMIT3173 Integrative Programming**

## **ASSIGNMENT 202605**

Student Name : Ong Kwong Wei

Student ID : 21WMR14117

Programme : Bachelor in Information Technology (Honours)  
(Information Security)

Tutorial Group : Group 4

System Title : LearnSync - Integrated Educational Resource and  
Collaborative Learning Portal

Chosen SDG : SDG 4 - Quality Education

Modules : Module 5 - Academic Progress Analytics and Evaluation

## AI Tools Usage Policy & Disclosure

This assignment is governed by the **TAR UMT Policy for the Use of Artificial Intelligence (AI) (PO/111:26)** and is classified under the **YELLOW (Limited AI)** category. AI tools may be used only for the purposes listed below, not to produce the technical work, analysis, or original content being assessed.

**Mandatory submission:** Every student must complete and submit the **AI Usage Disclosure Form** (below) with the report, whether or not AI tools were used, by 6<sup>th</sup> September 2026. A report submitted without the Form is incomplete and may not be marked.

### Permitted (Yellow) uses of AI tools:

- Language refinement (grammar, spelling, clarity) of text you wrote, without altering its technical meaning.
- Clarifying taught concepts (e.g. how a design pattern, web-service protocol, or attack works) to aid your understanding.
- Debugging help on code you wrote, where AI only locates or explains an error you then fix yourself.
- Brainstorming early ideas (e.g. SDG framings or topic angles), provided the chosen direction and its justification are your own.

### Prohibited uses of AI tools:

- Generating the PHP code, design-pattern implementation, class diagrams, or web-service code being assessed.
- Producing core arguments or analysis, including the design-pattern justification, threat analysis, and secure-coding rationale.
- Uploading confidential, proprietary, or others' personal data to public AI tools (Sections 2.5 and 5 of the AI Policy).

You remain responsible for the accuracy and originality of your work; AI errors do not excuse incorrect content. The lecturer may ask you to explain or demonstrate any part to verify authorship. Undisclosed or prohibited AI use is a breach of academic integrity under Section 4 of the AI Policy. **Complete the Form below; if AI was used, list each use and cite the tool(s) in your References (APA 7th).**

## AI Usage Disclosure Form

### Declaration (tick one):

- ☐ No AI tools were used in the preparation of this report.
- ☒ AI tools were used as declared in the table below.

| AI Tool Used (name & version) | Purpose / How It Was Used                                                                                                                          | Report Section(s) Affected   |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| Grammarly (2026 Edition)      | Used solely for language refinement, spelling verification, and grammatical sentence flow of my own written text without altering technical logic. | Sections 1, 2, 4.1, 4.3, 5.1 |

*If no AI tools were used, tick "No AI tools used". Non-disclosure breaches the AI Policy.*

I declare this Form is true and complete and that my AI use complied with the AI Policy and the Yellow conditions above.

Sign: \_\_\_\_\_ *Orig* \_\_\_\_\_

Date: \_\_\_\_\_ 6/9/2026 \_\_\_\_\_

# Table of Contents

|                                                                                                        |           |
|--------------------------------------------------------------------------------------------------------|-----------|
| <b>AI Tools Usage Policy &amp; Disclosure</b>                                                          | <b>2</b>  |
| <b>1. Introduction to the System</b>                                                                   | <b>4</b>  |
| 1.1 System Overview                                                                                    | 4         |
| 1.2 The Sustainable Development Goal                                                                   | 4         |
| 1.3 System Contribution to SDG 4 & Scope                                                               | 5         |
| <b>2. Module Description</b>                                                                           | <b>6</b>  |
| 2.1 Scope of Module 5: Academic Progress Analytics and Evaluation                                      | 6         |
| 2.2 Functional Breakdown & Class Paths                                                                 | 6         |
| F5.1: Setting an Assignment                                                                            | 6         |
| F5.2: The Submission Lifecycle                                                                         | 8         |
| F5.3: Marking and Writing the Grade                                                                    | 10        |
| F5.4: The Letter Grade Scale                                                                           | 11        |
| F5.5: Cohort Analytics                                                                                 | 12        |
| F5.6: The Completion Trend Chart, Drawn Through an XML Pipeline                                        | 13        |
| <b>3. Entity Classes</b>                                                                               | <b>15</b> |
| 3.1 Entity Class Diagram                                                                               | 15        |
| Figure 3.1: Entity Class Diagram for Module 5 Academic Progress Analytics and Evaluation               | 15        |
| 3.2 Entity Class Implementation (Eloquent ORM Mapping)                                                 | 16        |
| <b>4. Design Pattern</b>                                                                               | <b>19</b> |
| 4.1 Description of Design Pattern                                                                      | 19        |
| 4.2 Implementation of Design Pattern                                                                   | 21        |
| 1. The State Interface (app/Patterns/State/SubmissionState.php)                                        | 21        |
| 2. The Context Choosing Its State (app/Models/Submission.php)                                          | 21        |
| 3. Concrete State One, Draft (app/Patterns/State/DraftState.php)                                       | 23        |
| 4. Concrete State Two, Submitted (app/Patterns/State/SubmittedState.php)                               | 24        |
| 5. Concrete State Three, Graded (app/Patterns/State/GradedState.php)                                   | 25        |
| 6. The Controller, Which Never Checks the State Column (app/Http/Controllers/SubmissionController.php) | 26        |
| 4.3 Justification of Design Pattern                                                                    | 27        |
| <b>5. Software Security</b>                                                                            | <b>28</b> |
| 5.1 Potential Threat/Attack                                                                            | 28        |
| 5.2 Secure Coding Practice                                                                             | 30        |
| <b>6. Web Services</b>                                                                                 | <b>34</b> |
| 6.1 Web Service Exposure                                                                               | 34        |
| 6.2 Web Service Consumption                                                                            | 38        |
| <b>7. References</b>                                                                                   | <b>42</b> |
| <b>8. Appendices</b>                                                                                   | <b>43</b> |
| Appendix A: Automated Testing Results                                                                  | 43        |
| Appendix B: GitHub Repository URL                                                                      | 46        |

# 1. Introduction to the System

## 1.1 System Overview

LearnSync is a web based Learning Management System (LMS). Lecturers use it to upload notes, set quizzes, give assignments and mark student work. Students use it to study the notes, sit the quizzes, hand in assignments, watch their progress, and ask questions in a course forum.

The problem LearnSync solves is about proof. In most systems, the certificate a student earns is just a PDF file. Anyone can open that PDF in a word processor and change the marks. If an employer wants to check whether a certificate is real, they must email the college and wait days for a reply. Most employers do not bother.

LearnSync fixes this. Every certificate it creates carries a unique ID, a QR code anyone can scan, and a hidden security code proving the record has not been changed. Anyone can check a certificate in seconds, for free, without an account.

## 1.2 The Sustainable Development Goal

LearnSync is built around United Nations Sustainable Development Goal 4: Quality Education (SDG 4).

What SDG 4 aims to do:

SDG 4 wants to make sure education is fair, good quality and available to everyone by 2030 (United Nations, 2015). Target 4.3 is about giving everyone fair access to affordable college and university education. Target 4.4 is about increasing the number of people who have real skills for getting a job.

### 1.3 System Contribution to SDG 4 & Scope

LearnSync supports SDG 4 in three ways:

- Who benefits: Students who earn certificates, lecturers who teach and mark the courses, administrators who control access, and outside people such as employers who need to check a qualification.
- Showing a lecturer where a class is struggling: Module 5 turns marks into a picture. A lecturer looking at a list of eighty numbers cannot see anything useful. The same numbers as a class average, a grade distribution and a completion trend show immediately that half the class failed one particular assessment. SDG 4 is about quality of education, and a lecturer cannot improve what they cannot see.
- Making qualifications checkable: Every certificate carries a unique ID, a QR code and a security code, so an employer can confirm it instantly and for free.

What the system does not do: LearnSync does not replace official government accreditation, and the analytics are descriptive rather than predictive. The module reports what has happened. It does not try to predict which students will fail, because acting on a prediction about a person is a decision a lecturer should make, not a piece of software.

## 2. Module Description

### 2.1 Scope of Module 5: Academic Progress Analytics and Evaluation

I am the developer in charge of Module 5, the Academic Progress Analytics and Evaluation Module. I designed and built:

- assignments, with a brief, a due date and a per assignment policy on whether late work is accepted
- the submission lifecycle, where a student uploads a draft, may replace it, and then hands it in
- the lecturer's marking queue and the marking screen
- the grade record, which my module is the only writer of
- cohort analytics for lecturers and administrators, including a completion trend chart drawn through an XML pipeline
- the letter grade scale used everywhere in LearnSync

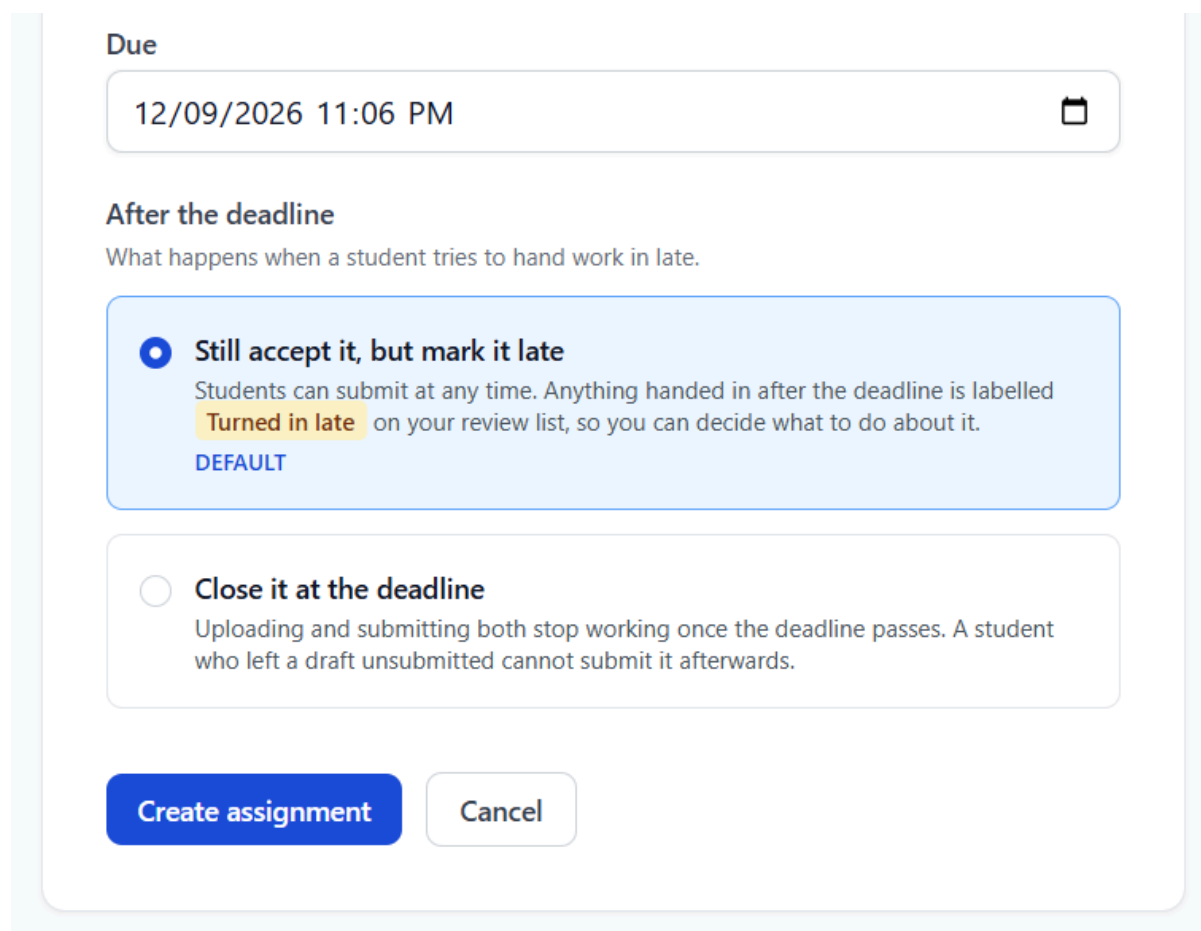
The submission lifecycle is why this module uses the State pattern. A submission behaves like a different object depending on where it is in its life, and that difference belongs in the object rather than in a chain of if statements spread across controllers.

One boundary is worth stating up front. My module is the only writer of the grades table. Module 1 reads grades to work out progress and to decide about certificates, but never writes one. That single write is also what triggers the whole credentialing chain.

### 2.2 Functional Breakdown & Class Paths

#### F5.1: Setting an Assignment

- Description: A lecturer creates an assignment on a course they own, giving it a title, a brief and a due date. They also choose whether late work is accepted. The default is to accept it and label it as turned in late, because a lecturer usually wants to see the work even when it is overdue, but the option exists to close an assignment firmly at its deadline.
- Class Paths:
  - Controller: `app/Http/Controllers/AssignmentController.php` (create, store, show, edit, update, destroy)
  - Model: `app/Models/Assignment.php`
  - View        Templates:        `resources/views/assignments/_form.blade.php`,  
                                 `resources/views/assignments/show.blade.php`



The image shows a web form for creating an assignment. At the top, there is a 'Due' date field set to '12/09/2026 11:06 PM' with a calendar icon. Below this is a section titled 'After the deadline' with the subtitle 'What happens when a student tries to hand work in late.' There are two radio button options. The first option, 'Still accept it, but mark it late', is selected and highlighted with a blue background. It includes a description: 'Students can submit at any time. Anything handed in after the deadline is labelled **Turned in late** on your review list, so you can decide what to do about it.' and the word 'DEFAULT' in blue. The second option is 'Close it at the deadline', which is not selected. It includes a description: 'Uploading and submitting both stop working once the deadline passes. A student who left a draft unsubmitted cannot submit it afterwards.' At the bottom of the form are two buttons: 'Create assignment' (blue) and 'Cancel' (white with a grey border).

Due

12/09/2026 11:06 PM

**After the deadline**

What happens when a student tries to hand work in late.

☒ **Still accept it, but mark it late**

Students can submit at any time. Anything handed in after the deadline is labelled **Turned in late** on your review list, so you can decide what to do about it.

DEFAULT

☐ **Close it at the deadline**

Uploading and submitting both stop working once the deadline passes. A student who left a draft unsubmitted cannot submit it afterwards.

Create assignment Cancel

Figure 2.1: The Assignment Form Showing the Late Submission Option

### F5.2: The Submission Lifecycle

- Description: A student uploads a file, which is saved as a draft. While it is a draft they may replace it as many times as they like. When they are ready they press Submit for marking, and the upload control disappears.

What a submission allows depends entirely on where it is in its life:

| State     | Student may replace the file | Student may submit | Lecturer may mark |
|-----------|------------------------------|--------------------|-------------------|
| Draft     | Yes                          | Yes                | No                |
| Submitted | No                           | No                 | Yes               |
| Graded    | No                           | No                 | No                |

The submission itself decides this, not the controller. Section 4 explains how.

Submitted files are stored on the private disk, which is not reachable from the web, and are served by a controller that checks the requester is either the student who submitted or the lecturer marking it.

- Class Paths:
  - Controller: app/Http/Controllers/SubmissionController.php (store, submit, grade, download)
  - Model: app/Models/Submission.php
  - States: app/Patterns/State/DraftState.php, SubmittedState.php, GradedState.php



File saved as a draft. Submit it when you are ready.

STATUS

**Draft**

Current file: [download](#)

The deadline passed 1 day ago. You can still submit, but it will be recorded as **turned in late**.

Replace file

[Choose File](#) No file chosen

[Save draft](#)

[Submit late for marking](#)

Figure 2.2: A Draft Submission, with the Replace File Control Available

STATUS

**Submitted** **TURNED IN LATE**

Current file: [download](#)

This submission is already submitted, so the file is locked.

Figure 2.3: The Same Submission After Handing In, with the Control Gone

### F5.3: Marking and Writing the Grade

- Description: The lecturer's dashboard carries a review queue of everything handed in and not yet marked. Opening a submission shows the student's file and a box for the mark.
- Assigning a mark writes a Grade. That one write is the event the whole credentialing chain hangs from: Module 1 listens for it, recalculates the student's progress, evaluates every badge rule and, if the student now qualifies, issues a certificate.
- Marking is refused unless the submission is in the submitted state. A draft has not been handed in, and an already graded submission must not be re-marked, because a certificate may already have been issued against the original mark.
- Class Paths:
  - Controller: app/Http/Controllers/SubmissionController.php (grade)
  - State: app/Patterns/State/SubmittedState.php (assignGrade)
  - Model: app/Models/Grade.php

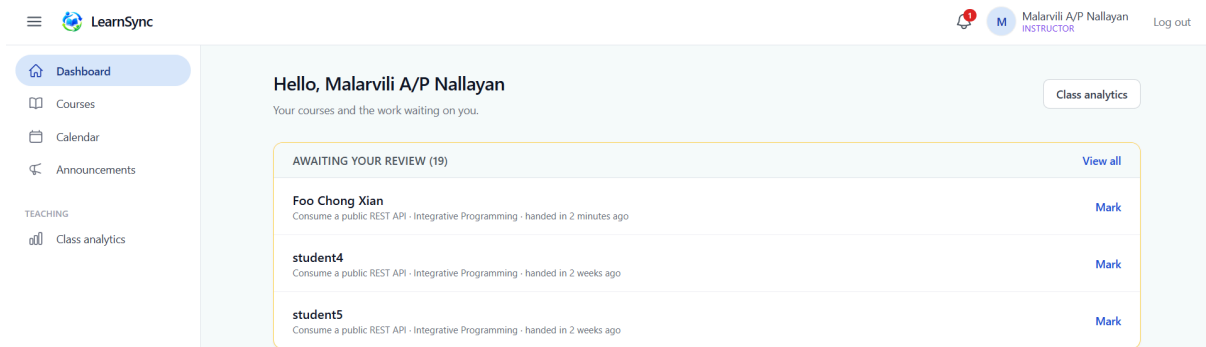


Figure 2.4: The Lecturer's Review Queue on the Dashboard

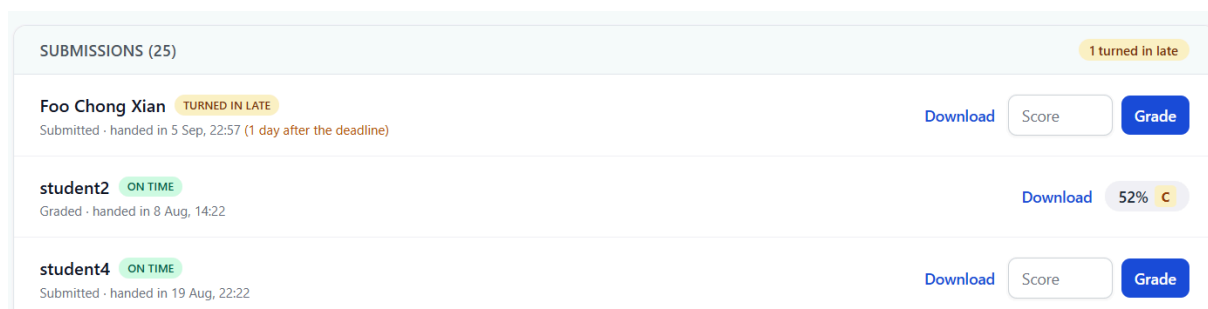


Figure 2.5: The Marking Screen with the Score Box

### F5.4: The Letter Grade Scale

- Description: Marks are stored as a percentage, and the letter is worked out from the percentage every time it is displayed rather than being saved alongside it. This means a mark and its letter can never disagree, and correcting the scale corrects every grade in the system at once.
- The scale runs from A at 80 and above down to F below 40, with the pass mark at 40. The analytics screen groups results into five families of A, B, C, D and F, because eleven separate bars would be unreadable while five tell the same story.
- Class Paths:
  - Support Class: `app/Support/GradeScale.php` (letterFor, pointFor, familyFor, isPass, legend)

← [Back to Integrative Programming](#)

## Consume a public REST API

Due 4 September 2026, 05:22 · deadline passed

Late submissions are accepted and marked "Turned in late".

Write a small client that calls a public REST API, parses the JSON response and displays it. Submit your source as a zip archive with a short report.

STATUS

**Graded** TURNED IN LATE

Current file: [download](#)

This submission is already graded, so the file is locked.

GRADE

**50%** C

Figure 2.6: The Grade Scale Legend Shown to Students

## F5.5: Cohort Analytics

- Description: For each course, the analytics screen reports the class average, the highest and lowest marks, how many passed, the grade distribution, how many submissions are waiting to be marked, and the average turnaround time between a student submitting and the mark being recorded.
- A lecturer sees only their own courses. An administrator sees all of them. This is decided by permission key rather than by comparing roles.
- The figures are all about a cohort. Nothing on this screen is about motivating an individual student, because a student's own progress towards their next certificate belongs to Module 1.
- Class Paths:
  - Controller: `app/Http/Controllers/AnalyticsController.php` (index, statisticsFor, distribution, averageTurnaroundHours)
  - View Template: `resources/views/analytics/index.blade.php`

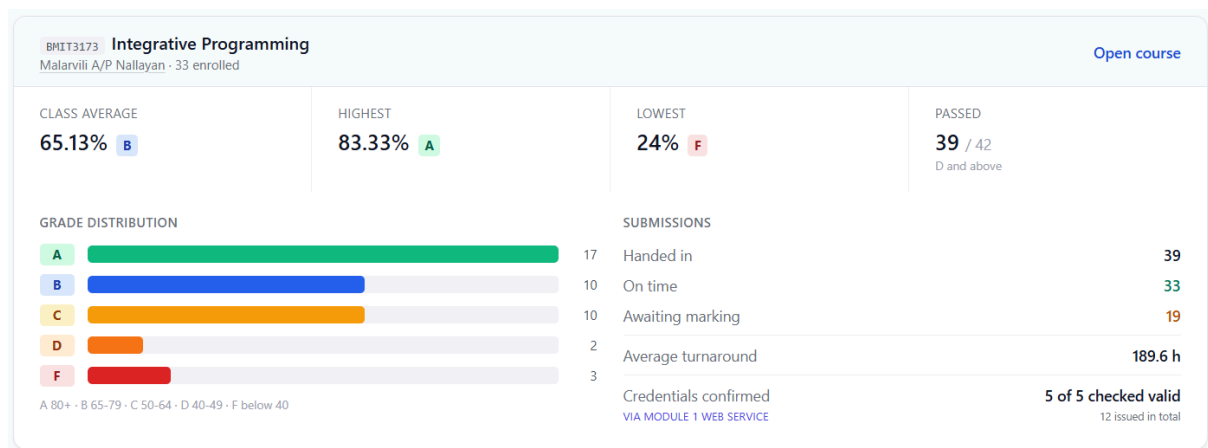


Figure 2.7: The Analytics Screen Showing Averages and the Grade Distribution

### F5.6: The Completion Trend Chart, Drawn Through an XML Pipeline

- Description: The completion trend chart is produced by an XML pipeline rather than a JavaScript charting library. The figures are read with Eloquent, built into an XML document with DOMDocument, validated against an XSD schema, and transformed into an SVG image by an XSLT stylesheet.
- The pipeline in one line is: Eloquent, then DOMDocument, then XSD validation, then XSLT, then SVG.
- This approach was chosen because SVG is itself an XML vocabulary, which makes drawing the chart a genuine XML to XML transformation rather than an exercise invented to demonstrate one. The same document is downloadable at `/analytics/export.xml`, so it is a real data export rather than a throwaway step.
- The schema is enforced rather than decorative. Percentages are a decimal restricted to 0 through 100, counts are non negative integers, dates are typed, and the grade letter is an enumeration of exactly A, B, C, D and F. A document carrying an average of 150 is rejected. Validation failure is not fatal: the error is logged and the page renders without the chart, so a schema fault can never take down a working analytics screen.
- Class Paths:
  - Controller: `app/Http/Controllers/AnalyticsController.php` (buildXml, validates, renderChart, exportXml)
  - Schema: `resources/xml/analytics.xsd`
  - Stylesheet: `resources/xml/analytics-chart.xsl`

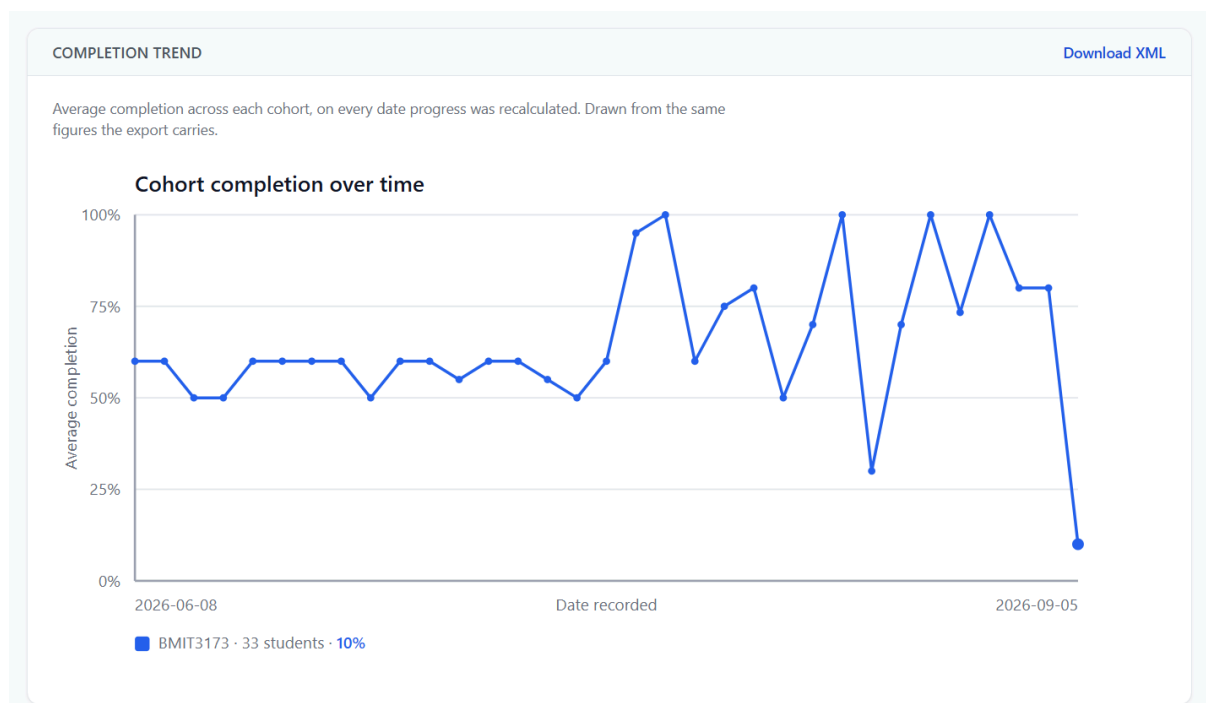


Figure 2.8: The Completion Trend Chart Rendered as SVG

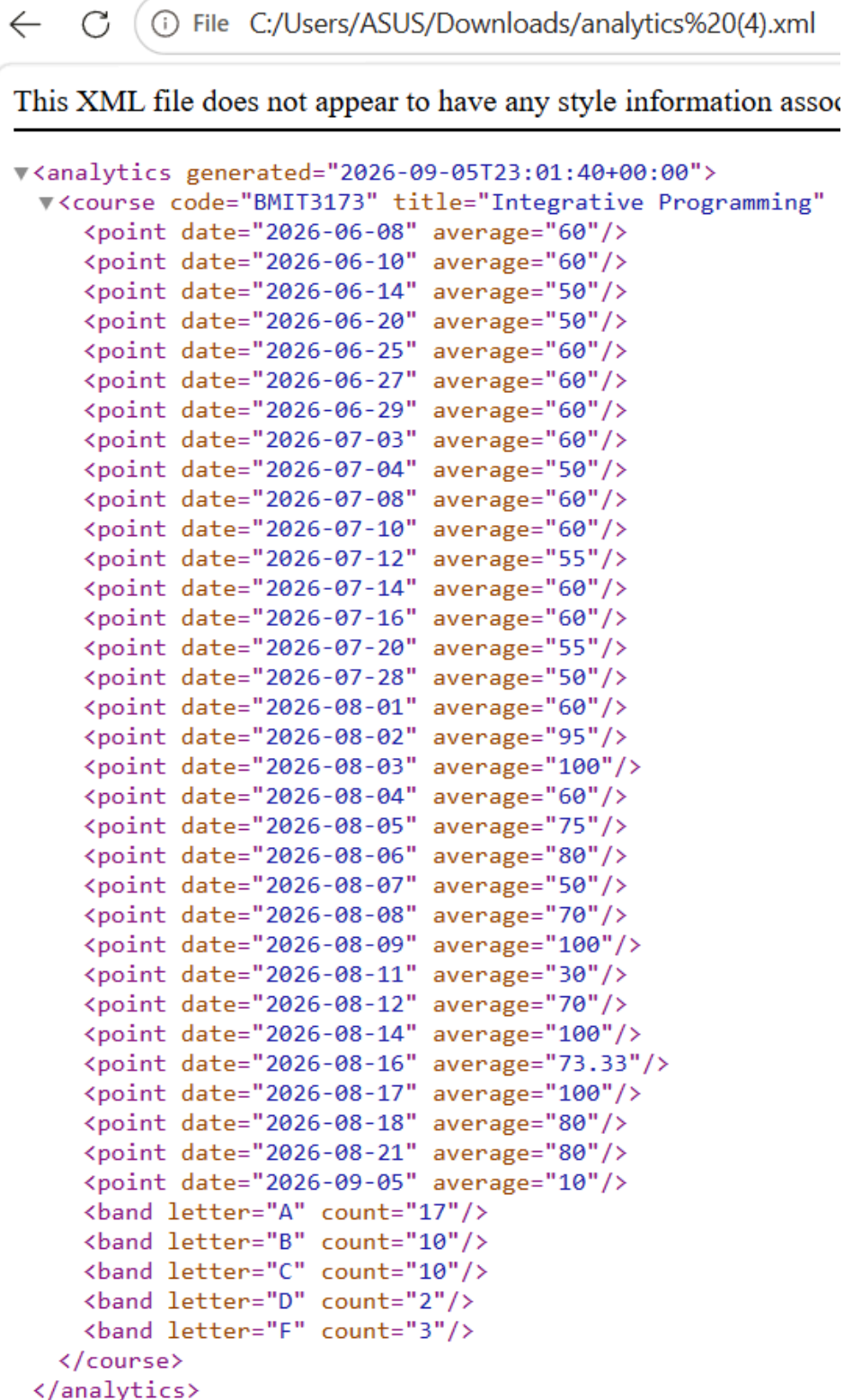


Figure 2.9: The Same Data Downloaded as XML from /analytics/export.xml

### 3. Entity Classes

#### 3.1 Entity Class Diagram

The diagram below shows the entity classes using object references, meaning one object points to another object, instead of database foreign keys.

Diagram link:

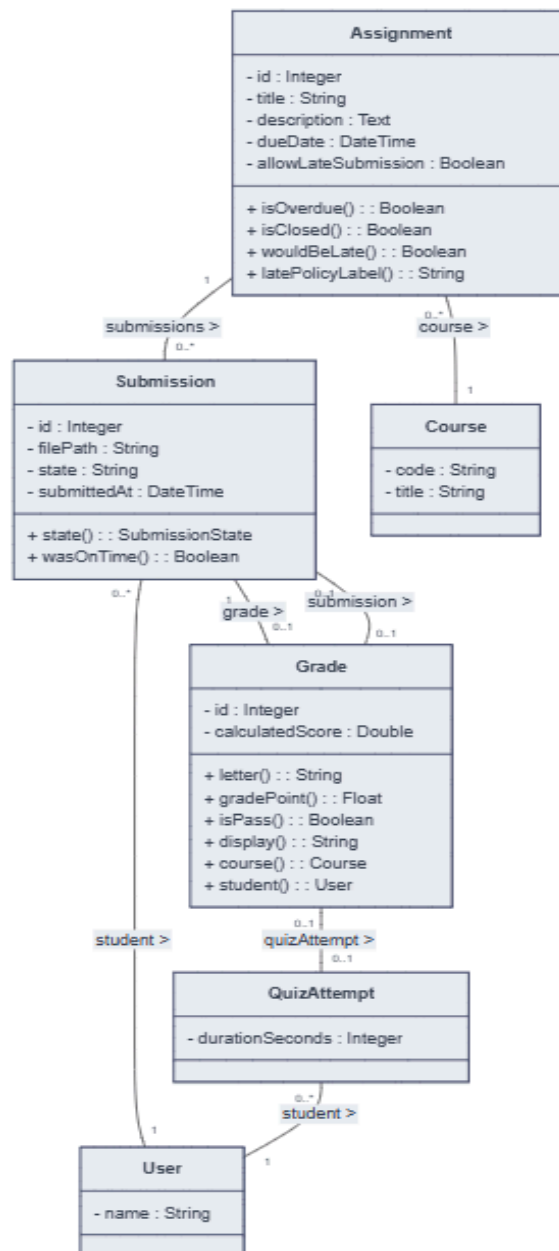


Figure 3.1: Entity Class Diagram for Module 5 Academic Progress Analytics and Evaluation

### 3.2 Entity Class Implementation (Eloquent ORM Mapping)

The classes are written in PHP using Laravel's Eloquent ORM. Relationships are written as methods, so the code never writes SQL. Calling `$submission->student` returns a whole User object, so `$submission->student->name` reads the student's name by following the link between objects.

```
namespace App\Models;

use App\Patterns\State\DraftState;
use App\Patterns\State\GradedState;
use App\Patterns\State\SubmissionState;
use App\Patterns\State\SubmittedState;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasOne;

/**
 * Module 5 -- the CONTEXT of the State pattern.
 *
 * The model holds the data and hands every lifecycle decision to its state
 * object. Nothing outside app/Patterns/State decides whether an edit or a
 * grade is allowed.
 */
class Submission extends Model
{
    // The state column's value mapped to the class that implements it
    private const STATES = [
        'draft'    => DraftState::class,
        'submitted' => SubmittedState::class,
        'graded'   => GradedState::class,
    ];

    protected $fillable = [
        'assignment_id', 'student_id', 'file_path', 'state', 'submitted_at',
    ];

    protected function casts(): array
    {
        return ['submitted_at' => 'datetime'];
    }

    /**
     * The state object for this submission's current state.
     *
     * An unknown value falls back to draft, so a hand edited database row
     * can never leave a submission with no behaviour at all.
     */
    public function state(): SubmissionState
    {
        $class = self::STATES[$this->state] ?? DraftState::class;
    }
}
```



```

    return new $class();
}

/** Object reference: a submission answers one assignment */
public function assignment(): BelongsTo
{
    return $this->belongsTo(Assignment::class);
}

/** Object reference: a submission belongs to one student */
public function student(): BelongsTo
{
    return $this->belongsTo(User::class, 'student_id');
}

/** Object reference: a submission may carry one mark */
public function grade(): HasOne
{
    return $this->hasOne(Grade::class);
}

/** Feeds the on time badge rule owned by Module 1 */
public function wasOnTime(): bool
{
    if ($this->submitted_at === null) {
        return false;
    }

    return $this->submitted_at->lessThanOrEqualTo($this->assignment->due_date);
}
}

```

```

namespace App\Models;

use App\Support\GradeScale;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

/**
 * Module 5 -- the authoritative mark.
 *
 * Module 5 is the only writer. Module 1 reads these as input to progress and
 * certificate decisions and never writes one.
 */
class Grade extends Model
{
    protected $fillable = ['submission_id', 'quiz_attempt_id', 'calculated_score'];

    protected function casts(): array
    {

```

```

    return ['calculated_score' => 'double'];
}

/** Object reference: the coursework this mark is for, if any */
public function submission(): BelongsTo
{
    return $this->belongsTo(Submission::class);
}

/** Object reference: the quiz attempt this mark is for, if any */
public function quizAttempt(): BelongsTo
{
    return $this->belongsTo(QuizAttempt::class);
}

// The letter is derived from the scale rather than stored, so a mark and
// its letter can never drift apart.
public function letter(): string
{
    return GradeScale::letterFor($this->calculated_score);
}

/**
 * The course this mark belongs to, whichever route it came in by.
 * Module 1 uses this to know which course to recalculate.
 */
public function course(): ?Course
{
    return $this->submission?->assignment?->course
        ?? $this->quizAttempt?->quiz?->course;
}
}

```

## 4. Design Pattern

### 4.1 Description of Design Pattern

For Module 5, I used the State Design Pattern, which is a Gang of Four Behavioural Pattern.

#### What the pattern means:

Gamma, Helm, Johnson and Vlissides (1994) describe the State Pattern as one that lets an object change its behaviour when its internal state changes, so that the object appears to change its class.

In simple words, some objects behave completely differently depending on what has happened to them so far. The usual way to write this is to store the situation in a column and check that column before doing anything. Every method starts with an if statement asking what state we are in, and every new state means going back and adding another branch to every one of those methods. The rules end up scattered across many places and it is easy to miss one.

The State pattern moves each situation into its own class. All those classes offer the same methods, and the object holds one of them. Asking the object to do something asks the current state object, and the answer changes because the object being asked has changed.

#### How this works in my module:

A submission behaves like a different thing depending on where it is in its life. In draft, the file can be replaced freely. Once submitted, it is locked and waiting. Once graded, nothing may change at all.

- Context: the Submission model. It holds the data and delegates every decision.
- State interface: SubmissionState, defining what any state must be able to answer.
- Concrete states: DraftState, SubmittedState and GradedState.
- Refusal: IllegalSubmissionTransition, an exception thrown by a state asked to do something it does not allow.

The three states and what each permits:

|                | canUpdateFile() | canSubmit() | canAssignGrade() |
|----------------|-----------------|-------------|------------------|
| DraftState     | true            | true        | false            |
| SubmittedState | false           | false       | true             |
| GradedState    | false           | false       | false            |

The states also move the submission along. DraftState::submit() sets the state to submitted and stamps the time. SubmittedState::assignGrade() writes the mark and sets the state to

graded. The object changes its own state as work progresses, which is what separates State from Strategy.

How State differs from Strategy:

Module 4 uses Strategy, and the two patterns look similar because both hold an object and call a method on it instead of using an if statement. The difference is who decides which object and whether it changes.

With Strategy, the caller picks the algorithm and it stays fixed for the whole job. A multiple choice question is marked by the multiple choice strategy from beginning to end, and the strategy never swaps itself for another one.

With State, the object picks its own state from its stored data, and the state changes as work progresses. A submission that was a draft a moment ago is a submitted one now, because `DraftState::submit()` moved it. The object appears to change its class over time.

Diagram link:

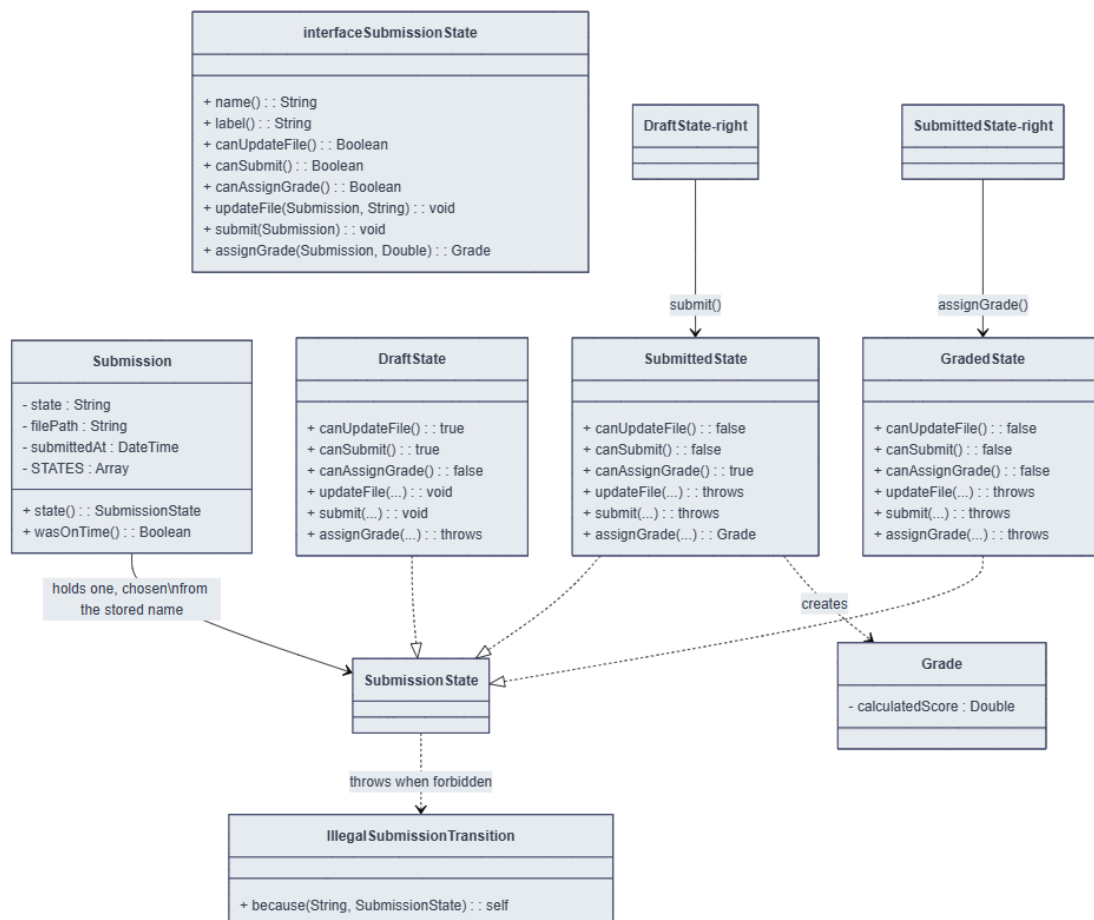


Figure 4.1: State Pattern Class Diagram for the Submission Lifecycle

## 4.2 Implementation of Design Pattern

### 1. The State Interface (app/Patterns/State/SubmissionState.php)

```

namespace App\Patterns\State;

use App\Models\Grade;
use App\Models\Submission;

/**
 * A Submission behaves differently depending on where it is in its lifecycle,
 * and the State pattern puts that difference in the state object rather than
 * in a chain of if statements spread across controllers.
 *
 * Draft    -- the student may re-upload freely; nothing may be graded yet
 * Submitted -- locked from edits, waiting for the lecturer
 * Graded   -- final; neither the file nor the mark may change
 *
 * Every transition and every permission question goes through the state
 * object, so a controller can never accidentally let a student edit work
 * that has already been marked.
 */
interface SubmissionState
{
    /** The value stored in submissions.state for this state. */
    public function name(): string;

    /** Human readable label for the interface. */
    public function label(): string;

    /** May the student replace the uploaded file? */
    public function canUpdateFile(): bool;

    /** May the student hand the work in from here? */
    public function canSubmit(): bool;

    /** May a lecturer put a mark against it? */
    public function canAssignGrade(): bool;

    /** @throws IllegalSubmissionTransition when this state forbids it */
    public function updateFile(Submission $submission, string $path): void;

    /** @throws IllegalSubmissionTransition when this state forbids it */
    public function submit(Submission $submission): void;

    /** @throws IllegalSubmissionTransition when this state forbids it */
    public function assignGrade(Submission $submission, float $score): Grade;
}

```

## 2. The Context Choosing Its State (app/Models/Submission.php)

```
class Submission extends Model
{
    private const STATES = [
        'draft'    => DraftState::class,
        'submitted' => SubmittedState::class,
        'graded'   => GradedState::class,
    ];

    /**
     * The state object for this submission's current state.
     *
     * An unknown value falls back to draft, so a hand edited database row
     * can never leave a submission with no behaviour at all.
     */
    public function state(): SubmissionState
    {
        $class = self::STATES[$this->state] ?? DraftState::class;

        return new $class();
    }
}
```

### 3. Concrete State One, Draft (app/Patterns/State/DraftState.php)

```

namespace App\Patterns\State;

use App\Models\Grade;
use App\Models\Submission;

/**
 * The student is still working. They may re-upload as often as they like,
 * and nothing can be marked yet.
 */
class DraftState implements SubmissionState
{
    public function name(): string { return 'draft'; }
    public function label(): string { return 'a draft'; }

    public function canUpdateFile(): bool { return true; }
    public function canSubmit(): bool { return true; }
    public function canAssignGrade(): bool { return false; }

    public function updateFile(Submission $submission, string $path): void
    {
        $submission->update(['file_path' => $path]);
    }

    // THE STATE MOVES THE OBJECT ALONG. This is what makes it State and
    // not Strategy: the object does not stay in this state afterwards.
    public function submit(Submission $submission): void
    {
        $submission->update([
            'state' => (new SubmittedState())->name(),
            // Stamped here rather than derived from updated_at, because the
            // on time badge rule compares this against the due date.
            'submitted_at' => now(),
        ]);
    }

    public function assignGrade(Submission $submission, float $score): Grade
    {
        // The refusal lives in the state, not in a controller's if statement
        throw IllegalSubmissionTransition::because('assign a grade', $this);
    }
}

```

**4. Concrete State Two, Submitted (app/Patterns/State/SubmittedState.php)**

```

namespace App\Patterns\State;

use App\Models\Grade;
use App\Models\Submission;

/**
 * Handed in and locked. The student can no longer touch the file; the
 * lecturer can now mark it.
 */
class SubmittedState implements SubmissionState
{
    public function name(): string { return 'submitted'; }
    public function label(): string { return 'already submitted'; }

    public function canUpdateFile(): bool { return false; }
    public function canSubmit(): bool { return false; }
    public function canAssignGrade(): bool { return true; }

    public function updateFile(Submission $submission, string $path): void
    {
        throw IllegalSubmissionTransition::because('replace the file', $this);
    }

    public function submit(Submission $submission): void
    {
        throw IllegalSubmissionTransition::because('submit again', $this);
    }

    /**
     * Write the authoritative Grade and move to graded.
     *
     * Module 5 is the sole writer of `grades`. Creating this row is what
     * wakes the credentialing chain in Module 1.
     */
    public function assignGrade(Submission $submission, float $score): Grade
    {
        $grade = Grade::updateOrCreate(
            ['submission_id' => $submission->id],
            ['calculated_score' => $score]
        );

        $submission->update(['state' => (new GradedState())->name()]);

        return $grade;
    }
}

```



**5. Concrete State Three, Graded (app/Patterns/State/GradedState.php)**

```
namespace App\Patterns\State;

/**
 * Final. Neither the work nor the mark may change.
 *
 * This is the state that protects the credentialing chain: a certificate is
 * issued off the back of a grade, so letting a graded submission be edited
 * afterwards would leave a credential attesting to something that no longer
 * matches the record.
 */
class GradedState implements SubmissionState
{
    public function name(): string { return 'graded'; }
    public function label(): string { return 'already graded'; }

    public function canUpdateFile(): bool { return false; }
    public function canSubmit(): bool { return false; }
    public function canAssignGrade(): bool { return false; }

    public function updateFile(Submission $submission, string $path): void
    {
        throw IllegalSubmissionTransition::because('replace the file', $this);
    }

    public function submit(Submission $submission): void
    {
        throw IllegalSubmissionTransition::because('submit', $this);
    }

    public function assignGrade(Submission $submission, float $score): Grade
    {
        throw IllegalSubmissionTransition::because('re-grade', $this);
    }
}
```

## 6. The Controller, Which Never Checks the State Column (app/Http/Controllers/SubmissionController.php)

```
/**
 * The lecturer marks the work. The state moves submitted to graded and
 * writes the authoritative Grade, which triggers the CredentialAuthority.
 */
public function grade(Request $request, Submission $submission): RedirectResponse
{
    $assignment = $submission->assignment;

    abort_unless($request->user()->can('grade.assign'), 403);
    abort_unless($assignment->course->instructor_id === $request->user()->id, 403);

    $data = $request->validate([
        'calculated_score' => ['required', 'numeric', 'min:0', 'max:100'],
    ]);

    try {
        // NO if statement about the state column. The submission is asked,
        // and it refuses by throwing if the request is not legal.
        $submission->state()->assignGrade($submission, (float) $data['calculated_score']);
    } catch (IllegalSubmissionTransition $e) {
        return back()->with('error', $e->getMessage());
    }

    return back()->with('success', 'Grade recorded.');
```

The view asks the same object what to show, so the interface and the rules can never disagree:

```
{{-- resources/views/assignments/show.blade.php --}}
```

```
@if ($submission->state()->canUpdateFile())
    {{-- The upload control only exists when the state allows it --}}
    <form method="post" enctype="multipart/form-data" ...>
        @csrf
        <input type="file" name="file" required>
        <button type="submit">Replace file</button>
    </form>
@endif
```

### 4.3 Justification of Design Pattern

1. The rules would otherwise be scattered and would drift apart. Without the pattern, the same question, which is whether this submission may still be edited, would be asked in the upload method, the submit method, the marking method and the Blade template. Four copies of one rule. The first time somebody changed one and missed another, the interface would offer a button the server then refused, or worse, the server would allow something the interface had correctly hidden. Holding the rule in one class means there is only ever one copy to change.
2. The object refuses illegal work itself, rather than trusting the caller to check first. `GradedState::updateFile()` throws. It does not return false and hope somebody looks. This means a new controller method written next year, by somebody who has never read this report, still cannot let a student edit graded work. The safety does not depend on the next developer remembering the rule, which is the difference between a design that is safe and a design that is merely correct today.
3. It protects the credentialing chain, which is the highest value thing in the system. A certificate is issued off the back of a grade. If a graded submission could be edited or re-marked afterwards, a credential would exist attesting to something that no longer matched the record, and Module 1's integrity hash would then report the certificate as tampered even though nobody attacked it. `GradedState` refusing all three operations is what keeps that from happening.
4. Adding a fourth state would be adding a class, not editing many methods. A returned for resubmission state, letting a lecturer send work back, would mean writing one new class implementing the interface and adding one line to the map in `Submission::STATES`. No controller and no other state would change. That is the Open Closed Principle.
5. It follows the Single Responsibility Principle, and it is testable. `SubmissionController::grade()` has one job: check the lecturer is allowed, validate the mark, ask the submission to record it. Deciding whether recording is legal is a different job living in its own class, and each state class can be constructed directly in a test without any web request at all.

## 5. Software Security

### 5.1 Potential Threat/Attack

#### Threat 1: SQL Injection Through Analytics and Marking Queries (OWASP A03: Injection)

##### Attack Description:

SQL injection happens when data typed by a user is joined onto a SQL query as text, so the database ends up treating part of that data as instructions rather than as a value.

My module is where this threat is most serious in LearnSync, for two reasons. First, it runs the heaviest queries in the system: the analytics screen filters marks by course, gathers submissions, and groups results, all driven by identifiers taken from the URL. Second, it is the module that reads and writes the grades table, which is the most valuable table in the application.

If a query were built by joining text together, such as:

```
// This is NOT what the code does. It shows the vulnerability being avoided.  
$results = DB::select("SELECT * FROM grades WHERE submission_id = " . $request->id);
```

then a request carrying `id=1 OR 1=1` would return every grade in the college. Worse, because MySQL accepts several statements in some configurations, a value such as `1; UPDATE grades SET calculated_score = 100` would change marks rather than merely read them. And since a grade write triggers the certificate machinery in Module 1, an injected UPDATE could end in genuine certificates being issued.

A subtler version of the same attack targets the analytics grouping. If a sort column or a grouping field were taken from a query string and pasted into the SQL, an attacker could use a UNION to append their own SELECT and read the users table, including the password hashes, through what looks like an ordinary chart.

##### Risk Impact:

Disclosure of every mark in the institution, unauthorised modification of grades leading to fraudulently issued certificates, and potentially disclosure of password hashes from unrelated tables through a UNION attack.

## Threat 2: Mass Assignment of Protected Grade and Submission Fields (OWASP A08: Software and Data Integrity Failures)

### Attack Description:

Mass assignment is the vulnerability that comes from convenience. Laravel lets a whole request be handed to a model in one line, such as `Submission::create($request->all())`. That is quick to write, and it means every field the attacker chose to send is written to the database, including fields the form never displayed.

My module has exactly the columns an attacker would want. A submission carries state and `submitted_at`. A grade carries a `calculated_score`. None of these appear as inputs on any form a student sees, but a student does not have to use the form. They can add fields to the request with the browser's developer tools or with a tool such as Postman.

The attack has three useful shapes here:

**Changing the state.** A student uploading a draft adds `state=graded` to the request. If the model accepted it, the submission would jump straight to graded without a lecturer ever looking at it, and would then be locked against further editing by my own State pattern, which the attacker is using as cover.

**Backdating a submission.** A student submitting late adds `submitted_at=2026-08-01 09:00:00`. The stored time would then be before the deadline, and `wasOnTime()` would report the work as punctual. That also feeds the on time submissions badge rule owned by Module 1, so the student collects an award they did not earn.

**Writing their own mark.** The most direct version: adding `calculated_score=100` to any request that creates or updates a grade. Because a grade write wakes the credentialing chain, this ends in a real certificate.

What makes mass assignment dangerous is that nothing looks wrong. The request is well formed, the validation rules pass because they only examine the fields they were told about, and the extra field simply rides along.

### Risk Impact:

Students awarding themselves marks, marking their own work as graded without a lecturer, and falsifying submission times to earn badges. All of these flow into the certificate machinery, so the end result is credentials that are cryptographically valid and academically false.

## 5.2 Secure Coding Practice

### Secure Practice 1: Every Query Goes Through the ORM, With Values Bound Rather Than Joined

OWASP Category: Database Security. There is no raw SQL anywhere in this application. Every query in my module is written through Eloquent, which sends the SQL and the values to the database separately, so a value can never be read as an instruction.

```
// app/Http/Controllers/AnalyticsController.php

/**
 * Every grade earned in a course, from both quizzes and coursework.
 *
 * SECURITY (Module 5): $course->quizzes() and $course->assignments() are
 * Eloquent relationships, and whereIn() binds its values. The course id
 * never becomes part of the SQL text, so it cannot alter the query's
 * meaning no matter what it contains.
 */
private function scoresFor(Course $course)
{
    $quizScores = Grade::whereIn('quiz_attempt_id', function ($query) use ($course) {
        $query->select('id')->from('quiz_attempts')
            ->whereIn('quiz_id', $course->quizzes()->select('id'));
    }->pluck('calculated_score'));

    $submissionScores = Grade::whereIn('submission_id', function ($query) use ($course) {
        $query->select('id')->from('submissions')
            ->whereIn('assignment_id', $course->assignments()->select('id'));
    }->pluck('calculated_score'));

    return $quizScores->merge($submissionScores);
}
```

How binding actually protects the query. Eloquent sends the database a prepared statement, in which the values are replaced by placeholders, and then sends the values separately. The database compiles the statement's structure before it ever sees the data. A value of 1 OR 1=1 is therefore searched for as a literal string, and matches nothing, because by the time it arrives the query's meaning is already fixed. This is stronger than escaping, which tries to neutralise dangerous characters and can be defeated by encoding tricks. Binding does not attempt to clean the input at all. It simply never lets it be code.

Route model binding closes the same door earlier. A controller signature of `grade(Request $request, Submission $submission)` means Laravel has already fetched that submission by its primary key before my method runs, using a bound query. A URL of `/submissions/1 OR 1=1/grade` does not reach my code at all; it fails to match the route and returns 404.

This is verifiable across the whole project. Searching the entire `app/` directory for `DB::raw`, `DB::select`, `DB::statement`, `DB::table`, `whereRaw`, `selectRaw`, `havingRaw` and `orderByRaw` returns no results in executable code. The only match is inside a comment recording that an earlier `whereRaw` was removed for exactly this reason:

```
// app/Http/Controllers/EnrolmentController.php

/*
 * Matched case-insensitively: the code is read off a slide or a chat
 * message, and rejecting a correct one over its capitalisation would be a
 * puzzle rather than a safeguard. The column collation (utf8mb4_unicode_ci)
 * does that comparison, so this stays a plain Eloquent where() -- an earlier
 * whereRaw('lower(...)') here was in breach of it.
 */
$course = Course::where('class_code', trim($data['class_code']))->first();
```

This is worth showing because it demonstrates the rule being enforced rather than merely stated. A case insensitive match is exactly the situation where a developer reaches for raw SQL, and the solution here was to let the database's own collation do the comparison instead.

## Secure Practice 2: An Explicit Allow List of Writable Fields on Every Model

OWASP Category: Data Protection. Every model in the application declares \$fillable, naming exactly which columns a mass assignment may write. Anything not on the list is silently ignored, no matter what the request contains.

```
// app/Models/Grade.php

/**
 * SECURITY (Module 5): only these three columns may be written by mass
 * assignment. There is no route by which a request field could set
 * anything else on a grade.
 */
protected $fillable = ['submission_id', 'quiz_attempt_id', 'calculated_score'];
```

```
// app/Models/Submission.php

/**
 * SECURITY (Module 5): `state` and `submitted_at` ARE on this list, because
 * the State classes must be able to write them when a submission moves
 * along its lifecycle. What protects them is not the fillable list but the
 * fact that no controller ever passes user input into these fields. Every
 * write of them is a literal value chosen by the state object itself.
 */
protected $fillable = [
    'assignment_id', 'student_id', 'file_path', 'state', 'submitted_at',
];
```

The second layer is that no controller in this module ever hands a whole request to a model. Each field is named individually, and the sensitive ones are set from values the server chose rather than values the request supplied:

```
// app/Http/Controllers/SubmissionController.php

// SECURITY (Module 5): the request is NEVER passed wholesale. Only two
// columns are named here, both derived from the authenticated session and
// the route, not from the request body. A student adding state=graded or
// calculated_score=100 to this request achieves nothing, because neither
// field is read.
$submission = Submission::firstOrCreate(
    ['assignment_id' => $assignment->id, 'student_id' => $request->user()->id],
    ['state' => 'draft']
);

$path = $request->file('file')->store('submissions/'.$assignment->id, 'local');

// The state object writes the file path. The student cannot choose it.
$submission->state()->updateFile($submission, $path);
```



```
// app/Http/Controllers/SubmissionController.php

// SECURITY (Module 5): the mark is validated to a numeric range first, and
// then passed as a single typed argument rather than as an array of request
// fields. There is no way for an extra field to travel alongside it.
$data = $request->validate([
    'calculated_score' => ['required', 'numeric', 'min:0', 'max:100'],
]);

$submission->state()->assignGrade($submission, (float) $data['calculated_score']);
```

Why the two layers are both needed. The \$fillable list is the safety net that catches a careless `create($request->all())` written in a hurry. Naming fields individually is the habit that means the net is never tested. Relying on only the first would leave the module one convenient line away from a hole, and relying on only the second would depend on every developer being careful for ever.

Why state is deliberately fillable, which looks wrong at first. The State classes write it, and they must be able to. What keeps it safe is that its value never comes from a request. `DraftState::submit()` writes `(new SubmittedState())->name()`, a literal produced by the code, and `submitted_at` is set to `now()` on the server. A student cannot influence either, because neither is read from their input at any point.

## 6. Web Services

Module 5 works with web services in both directions, and both are built and working in the code.

As a provider, it exposes `getCourseAnalytics`. Module 2 uses it to show a class performance summary on the course page, without needing to understand how a mark is worked out or repeating my grading rules.

As a consumer, it calls Module 1's `getCredentialStatus`. When reporting on a cohort, my module needs to know how many students hold a live credential. Rather than reading Module 1's certificates table and re-implementing its integrity checks, I ask Module 1 whether a credential is currently valid and take the answer.

I chose REST with JSON over SOAP because it is lighter, needs no WSDL contract file or XML wrapper, and can be called straight from a browser, from Postman, or from PHP.

Both halves follow one shared Interface Agreement held in `app/Support/Ifa.php`. Every request carries a `requestID` and a `timeStamp`. Every response carries a status of S, F or E, a `timeStamp`, and the `requestID` echoed back.

### 6.1 Web Service Exposure

#### Interface Agreement (IFA) for Service Exposure

|                      | Description                                                                                          |
|----------------------|------------------------------------------------------------------------------------------------------|
| Protocol             | RESTful Web Service (JSON over HTTP)                                                                 |
| Function Description | Returns cohort marks for one course: the average, highest, lowest, pass count and grade distribution |
| Source Module        | Module 5: Academic Progress Analytics and Evaluation Module                                          |
| Target Module        | Module 2 (Academic Resources Repository)                                                             |
| HTTP Method & URL    | GET <code>/api/analytics/course</code>                                                               |
| Controller Action    | <code>App\Http\Controllers\Api\AnalyticsApiController@courseAnalytics</code>                         |
| Function Name        | <code>getCourseAnalytics</code>                                                                      |
| Authentication       | Shared key in an X-API-Key header                                                                    |

#### Web Services Request Parameters (IFA Requirement)

| Field Name            | Field Type | Mandatory/Optional | Description              | Format               |
|-----------------------|------------|--------------------|--------------------------|----------------------|
| <code>courseId</code> | Integer    | Mandatory          | The course to report on. | Whole number above 0 |

|           |        |           |                                           |                                   |
|-----------|--------|-----------|-------------------------------------------|-----------------------------------|
| requestID | String | Mandatory | A unique ID so the request can be traced. | Letters, numbers, hyphens. Max 64 |
| timeStamp | String | Mandatory | The time the request was made.            | YYYY-MM-DDTHH:MM:SSZ              |

### Web Services Response Parameters (IFA Requirement)

| Field Name        | Field Type | Mandatory / Optional | Description                             | Format                                 |
|-------------------|------------|----------------------|-----------------------------------------|----------------------------------------|
| status            | String     | Mandatory            | Whether the request worked.             | S for Success, F for Fail, E for Error |
| timeStamp         | String     | Mandatory            | The time the answer was created.        | YYYY-MM-DDTHH:MM:SSZ                   |
| data.requestID    | String     | Mandatory            | The request ID sent back, for matching. | Letters, numbers, hyphens              |
| data.courseCode   | String     | Mandatory            | The course reported on.                 | Letters and numbers                    |
| data.gradedCount  | Integer    | Mandatory            | How many marks exist for the course.    | 0 or above                             |
| data.averageScore | Double     | Optional             | The class average.                      | 0.00 to 100.00                         |
| data.averageGrade | String     | Optional             | The letter for that average.            | A, A-, B+, B and so on                 |
| data.highestScore | Double     | Optional             | The best mark in the class.             | 0.00 to 100.00                         |
| data.lowestScore  | Double     | Optional             | The lowest mark in the class.           | 0.00 to 100.00                         |
| data.passCount    | Integer    | Optional             | How many marks were a pass.             | 0 or above                             |
| data.distribution | Object     | Optional             | Counts per letter family.               | {"A":17,"B":10,"C":10,"D":2,"F":2}     |

**Code Implementation (app/Http/Controllers/Api/AnalyticsApiController.php)**

```

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Course;
use App\Support\GradeScale;
use App\Support\Ifa;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Throwable;

/**
 * MODULE 5 exposes: getCourseAnalytics.
 *
 * SECURITY (Module 5): returns figures about a whole cohort and never about
 * a named individual. A service that returned per student marks would let
 * any key holder assemble a full transcript for somebody else, which the
 * summary shape avoids entirely.
 */
class AnalyticsApiController extends Controller
{
    public function courseAnalytics(Request $request): JsonResponse
    {
        // Ifa::baseRules() supplies the two fields every service demands,
        // so all five members validate them the same way.
        $validator = validator($request->all(), Ifa::baseRules() + [
            'courseId' => ['required', 'integer', 'min:1'],
        ]);

        if ($validator->fails()) {
            return Ifa::fail($request, ['errors' => $validator->errors()->all()]);
        }

        try {
            $course = Course::find($request->integer('courseId'));

            if ($course === null) {
                return Ifa::fail($request, [
                    'message' => 'No course exists with that ID.',
                ], 404);
            }

            $scores = $this->scoresFor($course);

            if ($scores->isEmpty()) {
                return Ifa::success($request, [
                    'courseCode' => $course->code,
                    'gradedCount' => 0,
                    'message' => 'No work has been marked in this course yet.',
                ]);
            }
        }
    }
}

```

```

    $average = round((float) $scores->avg(), 2);

    return Ifa::success($request, [
        'courseCode' => $course->code,
        'gradedCount' => $scores->count(),
        'averageScore' => $average,
        'averageGrade' => GradeScale::letterFor($average),
        'highestScore' => round((float) $scores->max(), 2),
        'lowestScore' => round((float) $scores->min(), 2),
        'passCount' => $scores->filter(fn ($s) => GradeScale::isPass($s))->count(),
        'distribution' => $this->distribution($scores),
    ]);

    } catch (Throwable $e) {
        Log::error('getCourseAnalytics failed', ['error' => $e->getMessage()]);

        // The exception message goes to the log, never to the caller.
        return Ifa::error($request);
    }
}
}
}

```

#### Real response from the running system:

```

{
  "status": "S",
  "timeStamp": "2026-09-05T16:30:35Z",
  "data": {
    "requestID": "ANL-REQ-99001",
    "courseCode": "BMIT3173",
    "gradedCount": 41,
    "averageScore": 65.9,
    "averageGrade": "B",
    "highestScore": 83.33,
    "lowestScore": 24,
    "passCount": 39,
    "distribution": { "A": 17, "B": 10, "C": 10, "D": 2, "F": 2 }
  }
}

```

## 6.2 Web Service Consumption

### Interface Agreement (IFA) for Service Consumption

|                              | Description                                                                     |
|------------------------------|---------------------------------------------------------------------------------|
| <b>Protocol</b>              | RESTful Web Service (JSON over HTTP)                                            |
| <b>Function Description</b>  | Returns the verification status of a credential, looked up by its credential ID |
| <b>Source Module</b>         | Module 1: Identity, Access and Digital Credentialing Module                     |
| <b>Consuming Module</b>      | Module 5: Academic Progress Analytics and Evaluation Module                     |
| <b>HTTP Method &amp; URL</b> | GET /api/credentials/verify                                                     |
| <b>Client Class</b>          | App\Support\Api\CredentialStatusClient@status                                   |
| <b>Function Name</b>         | getCredentialStatus                                                             |
| <b>Protocol</b>              | RESTful Web Service (JSON over HTTP)                                            |

### Web Services Request Parameters (IFA Requirement for Consumption)

| Field Name   | Field Type | Mandatory/Optional | Description                                        | Format                    |
|--------------|------------|--------------------|----------------------------------------------------|---------------------------|
| credentialId | String     | <b>Mandatory</b>   | The credential to check.                           | LS-YYYY-XXXXXX<br>XX      |
| detailFlag   | Integer    | <b>Mandatory</b>   | How much detail is wanted.                         | Module 5 always sends 1   |
| requestID    | String     | <b>Mandatory</b>   | A tracking ID made by Module 5, prefixed CRED-REQ. | Letters, numbers, hyphens |
| timeStamp    | String     | <b>Mandatory</b>   | The time the request was sent.                     | YYYY-MM-DDTHH:<br>MM:SSZ  |

**Web Services Response Parameters (IFA Requirement for Consumption)**

| Field Name            | Field Type | Mandatory/<br>Optional | Description                      | Format                                       |
|-----------------------|------------|------------------------|----------------------------------|----------------------------------------------|
| status                | String     | <b>Mandatory</b>       | Whether the request worked.      | S for Success, F for Fail, E for Error       |
| timeStamp             | String     | <b>Mandatory</b>       | The time the answer was created. | YYYY-MM-DDTHH:MM:SSZ                         |
| data.credentialStatus | String     | <b>Mandatory</b>       | The result of the check.         | VALID, REVOKED, TAMPERED, EXPIRED, NOT_FOUND |
| data.holderName       | String     | Optional               | Not requested by Module 5.       | Only returned at detailFlag 2 or above       |

**Consumption Code Implementation (app/Support/Api/CredentialStatusClient.php)**

```

namespace App\Support\Api;

/**
 * CONSUMES Module 1's getCredentialStatus service.
 *
 * Module 5 counts valid credentials in a cohort without needing to
 * understand integrity hashes or revocation. It asks Module 1 whether a
 * credential is currently valid and takes the answer.
 */
class CredentialStatusClient extends ServiceClient
{
    protected function requestPrefix(): string
    {
        return 'CRED-REQ';
    }

    /**
     * Whether a credential is genuine, and nothing about its holder.
     *
     * SECURITY (Module 5): detailFlag 1 is used on purpose. Module 5 is
     * counting credentials, so it has no business receiving names and
     * marks it does not need. Asking for the minimum is the right default.
     */
    public function status(string $credentialId): ?array
    {
        return $this->get('/credentials/verify', [
            'credentialId' => $credentialId,
            'detailFlag' => 1,
        ]);
    }

    /**
     * Is this credential currently valid?
     *
     * SECURITY (Module 5): returns false when the service cannot be
     * reached, which is the safe answer. An unreachable authority must
     * never be read as confirmation.
     */
    public function isValid(string $credentialId): bool
    {
        $data = $this->status($credentialId);

        return ($data['credentialStatus'] ?? null) === 'VALID';
    }
}

```

The fail safe default is the part worth pointing at. If Module 1 is down, isValid() returns false rather than true. A report that under counts credentials because a service was restarting is a small inaccuracy. A report that counts a revoked or forged credential as valid because nobody answered would be a lie, and would be worse than no report at all.



**Real response from the running system:**

```
{
  "status": "S",
  "timeStamp": "2026-09-05T16:30:16Z",
  "data": {
    "requestID": "CRED-REQ-8F2K1LM9QZ",
    "credentialStatus": "VALID"
  }
}
```

Figure 6.3: Postman Showing Module 5 Consuming Module 1's Credential Status Service

## 7. References

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

Laravel LLC. (2026). Laravel 12.x documentation: Eloquent ORM, mass assignment and query builder. <https://laravel.com/docs/12.x>

OWASP Foundation. (2021). OWASP Top 10:2021 The ten most critical web application security risks. Open Web Application Security Project. <https://owasp.org/Top10/>

OWASP Foundation. (2022). OWASP secure coding practices quick reference guide (Version 2.1). <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>

United Nations. (2015). Transforming our world: The 2030 agenda for sustainable development (Goal 4: Quality Education). United Nations Department of Economic and Social Affairs. <https://sdgs.un.org/goals/goal4>

W3C. (2004). XML Schema Part 2: Datatypes (2nd ed.). World Wide Web Consortium. <https://www.w3.org/TR/xmlschema-2/>

## 8. Appendices

### Appendix A: Automated Testing Results

```
c:\xampp\htdocs\edusystem>php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true 0.01s

PASS Tests\Feature\Auth\AuthenticationTest
✓ login screen can be rendered 0.59s
✓ users can authenticate using the login screen 0.08s
✓ users can not authenticate with invalid password 0.24s
✓ users can logout 0.03s

PASS Tests\Feature\Auth\EmailVerificationTest
✓ email verification screen can be rendered 0.04s
✓ email can be verified 0.04s
✓ email is not verified with invalid hash 0.03s

PASS Tests\Feature\Auth\PasswordConfirmationTest
✓ confirm password screen can be rendered 0.04s
✓ password can be confirmed 0.03s
✓ password is not confirmed with invalid password 0.23s

PASS Tests\Feature\Auth\PasswordResetTest
✓ reset password link screen can be rendered 0.03s
✓ reset password link can be requested 0.25s
✓ reset password screen can be rendered 0.26s
✓ password can be reset with valid token 0.28s

PASS Tests\Feature\Auth\PasswordUpdateTest
✓ password can be updated 0.04s
✓ correct password must be provided to update password 0.03s

PASS Tests\Feature\Auth\RegistrationTest
✓ open registration route does not exist 0.04s
✓ an unknown token is rejected 0.03s
✓ an expired invitation is refused 1.66s
✓ an already accepted invitation is refused 0.56s
✓ a valid invitation creates the account with its fixed role 0.05s
✓ a token cannot be redeemed twice 0.60s

PASS Tests\Feature\AwardAndActivityNotificationTest
✓ earning a certificate notifies the holder 0.53s
✓ earning a badge notifies the student 0.05s
✓ marking submitted work notifies the student 0.04s
✓ a marked quiz does not notify because the result is already on screen 0.03s
```

```

✓ inviting a student to a course notifies them 0.03s
✓ a switched off preference still suppresses the new types 0.03s

PASS Tests\Feature\AwardRuleTest
✓ an admin defined average score rule awards a badge 0.06s
✓ an admin defined quizzes completed rule counts distinct quizzes 0.05s
✓ an admin defined certificate rule mints a real credential 0.41s
✓ a certificate rule does not mint twice 0.56s
✓ a certificate rule is never handed out as a badge 0.50s
✓ deactivating a rule stops it without removing awards already made 0.06s

PASS Tests\Feature\CalendarEventDetailTest
✓ an enrolled student can open an events detail page 0.07s
✓ a student cannot open an event for a course they are not in 0.04s
✓ a lecturer cannot open an event for another lecturers course 0.04s
✓ an institution wide event is visible to everyone 0.05s
✓ a meeting shows a join button 0.05s
✓ an event with no link shows no join button 0.06s
✓ a malformed meeting link does not crash or render a button 0.06s
✓ a javascript url is never rendered as a join button 0.05s
✓ a student does not see the names of their classmates 0.06s

PASS Tests\Feature\CourseEnrolmentTest
✓ a student cannot leave a course themselves 0.05s
✓ the owning lecturer can remove a student 0.04s
✓ a lecturer cannot remove a student from another lecturers course 0.05s
✓ a student cannot remove a classmate 0.04s
✓ removing somebody who is not enrolled is a 404 0.04s

PASS Tests\Feature\CourseMaterialUploadTest
✓ a lecturer can upload a normal document 0.06s
✓ a php file is refused 0.04s
✓ a php file renamed to look like a pdf is still refused 0.05s
✓ an html file is refused 0.04s
✓ a javascript url cannot be saved as an external material 0.04s
✓ an ordinary external link is accepted 0.04s
✓ a lecturer cannot upload to another lecturers course 0.04s

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response 0.04s

PASS Tests\Feature\ProfileTest
✓ profile page is displayed 0.06s
✓ profile information can be updated 0.04s
✓ email verification status is unchanged when the email address is unchanged 0.04s
✓ user can delete their account 0.03s
✓ correct password must be provided to delete account 0.03s

PASS Tests\Feature\ProfileTest
✓ profile page is displayed 0.06s
✓ profile information can be updated 0.04s
✓ email verification status is unchanged when the email address is unchanged 0.04s
✓ user can delete their account 0.03s
✓ correct password must be provided to delete account 0.03s

PASS Tests\Feature\SubjectExpertBadgeTest
✓ passing every quiz in the subject awards the badge 0.06s
✓ attempting without passing does not award it 0.04s
✓ a subject with no quizzes awards nothing 0.03s
✓ resubmitting does not award a second copy 0.05s
✓ a quiz added afterwards does not revoke the badge 0.05s
✓ the badge is scoped to its own subject 0.06s

PASS Tests\Feature\WebServiceTest
✓ every response carries status timestamp and the request id 0.05s
✓ a request without the mandatory ifa fields is refused 0.03s
✓ an internal service refuses a caller with no api key 0.03s
✓ an internal service refuses a wrong api key 0.03s
✓ the credential service is public and needs no key 0.43s
✓ detail flag one does not disclose the holder 0.45s
✓ detail flag two returns the holder 0.54s
✓ an unknown credential reports not found rather than erroring 0.03s
✓ a tampered credential is reported over the service 0.45s
✓ the quiz service returns the best attempt 0.06s
✓ the analytics service returns cohort figures only 0.05s
✓ the notification service writes to the inbox 0.03s
✓ the notification service refuses a type outside the allow list 0.03s
✓ module 1 consumes module 2s course info service 0.08s
✓ module 5 consumes module 1s credential service 0.42s
✓ module 3 consumes module 4s quiz result service 0.05s
✓ a client returns null rather than throwing when the service is unreachable 1.07s

Tests: 86 passed (200 assertions)
Duration: 13.37s

```

Figure 8.1: Terminal Output of php artisan test Showing 86 Passing Tests

```

c:\xampp\htdocs\edusystem>php artisan test --filter=WebServiceTest

PASS Tests\Feature\WebServiceTest
✓ every response carries status timestamp and the request id 0.54s
✓ a request without the mandatory ifa fields is refused 0.03s
✓ an internal service refuses a caller with no api key 0.03s
✓ an internal service refuses a wrong api key 0.03s
✓ the credential service is public and needs no key 0.50s
✓ detail flag one does not disclose the holder 0.40s
✓ detail flag two returns the holder 0.40s
✓ an unknown credential reports not found rather than erroring 0.03s
✓ a tampered credential is reported over the service 0.39s
✓ the quiz service returns the best attempt 0.06s
✓ the analytics service returns cohort figures only 0.05s
✓ the notification service writes to the inbox 0.03s
✓ the notification service refuses a type outside the allow list 0.03s
✓ module 1 consumes module 2s course info service 0.06s
✓ module 5 consumes module 1s credential service 0.40s
✓ module 3 consumes module 4s quiz result service 0.04s
✓ a client returns null rather than throwing when the service is unreachable 1.04s

Tests: 17 passed (47 assertions)
Duration: 4.27s

```

Figure 8.2: Terminal Output of own modules php artisan test

## **Appendix B: GitHub Repository URL**

Team Repository: <https://github.com/NickFoo0924/edusystem>

Branch: master